Security Audit Report

Gno Updates

v1.0

January 9, 2026

Table of Contents

Table of Contents 2
License 4
Disclaimer 5
Introduction 6
Purpose of This Report 6
Codebase Submitted for the Audit 6
Methodology 8
Functionality Overview 8
How to Read This Report 9
Code Quality Criteria 10
Summary of Findings 11
Detailed Findings 13
1. Insufficient gas metering and unmetered operations lead to denial of service 13
2. Pass-by-value parameter in garbage collector nullifies gas accounting 14
3. ExpandWith omits memory accounting for wrapped values 15
4. Underestimated CPU cycle cost for doOpReturnAfterCopy 16
5. Unbounded recursion in Stacktrace serialization enables DoS 16
6. Type checker bypass for data structures and interfaces 17
7. Incorrect relations between interfaces can cause realm panics 17
8. Garbage collector can exhaust the call stack and crash the node 18
9. Integer overflow in memory allocator enables bypass of resource limits 19
10. Concerns regarding the DeepFill method 20
11. Incorrect early return in LeftmostX prevents proper heap escape analysis for nested selector expressions 21
12. Returning heap pointers across realms may enable phishing via metadata spoofing 21
13. Missing integrity check in exception chaining 22
14. Usage of undefined gno.MPStdlib and related identifiers cause compilation failures 23
15. Lack of transactional isolation in revive function 24
16. Usage of the type checker in testing mode 24
17. Nonce and package identifier are not validated during unmarshalling 25
18. Inconsistency in the single-linked list implementation 25
19. Missing functionality related to the autoNative parameter 26
20. Redundant code 26
21. Ineffective overflow protection 27
22. Missing inclusion of pckgName field in shallow size allocation calculation 28
23. Redundant slice length evaluation in defer stack handling 28
24. Use of the magic string "testing/base" reduces maintainability 29

25. Misleading naming 29
Appendix: Test Cases **30**
1. Test case for “Pass-by-value parameter in garbage collector nullifies gas accounting” 30
2. Test case for “ExpandWith omits memory accounting for wrapped values” 34
3. Test case for “Type checker bypass for data structures and interfaces” 37
4. Test case for “Incorrect relations between interfaces can cause realm panics” 37

License

THIS WORK IS LICENSED UNDER A CREATIVE COMMONS ATTRIBUTION-NODERIVATIVES 4.0 INTERNATIONAL LICENSE.

Disclaimer

THE CONTENT OF THIS AUDIT REPORT IS PROVIDED “AS IS”, WITHOUT REPRESENTATIONS AND WARRANTIES OF ANY KIND.

THE AUTHOR AND HIS EMPLOYER DISCLAIM ANY LIABILITY FOR DAMAGE ARISING OUT OF, OR IN CONNECTION WITH, THIS AUDIT REPORT.

THIS AUDIT REPORT WAS PREPARED EXCLUSIVELY FOR AND IN THE INTEREST OF THE CLIENT AND SHALL NOT CONSTRUE ANY LEGAL RELATIONSHIP TOWARDS THIRD PARTIES. IN PARTICULAR, THE AUTHOR AND HIS EMPLOYER UNDERTAKE NO LIABILITY OR RESPONSIBILITY TOWARDS THIRD PARTIES AND PROVIDE NO WARRANTIES REGARDING THE FACTUAL ACCURACY OR COMPLETENESS OF THE AUDIT REPORT.

FOR THE AVOIDANCE OF DOUBT, NOTHING CONTAINED IN THIS AUDIT REPORT SHALL BE CONSTRUED TO IMPOSE ADDITIONAL OBLIGATIONS ON COMPANY, INCLUDING WITHOUT LIMITATION WARRANTIES OR LIABILITIES.

COPYRIGHT OF THIS REPORT REMAINS WITH THE AUTHOR.

This audit has been performed by

Oak Security GmbH

https://oaksecurity.io/
info@oaksecurity.io

Introduction

Purpose of This Report

Oak Security GmbH has been engaged by NewTendermint LLC to perform a security audit of Gno Updates.

The objectives of the audit are as follows:

1. Determine the correct functioning of the protocol, in accordance with the project specification.
2. Determine possible vulnerabilities, which could be exploited by an attacker.
3. Determine smart contract bugs, which might lead to unexpected behavior.
4. Analyze whether best practices have been applied during development.
5. Make recommendations to improve code safety and readability.

This report represents a summary of the findings.

As with any code audit, there is a limit to which vulnerabilities can be found, and unexpected execution paths may still be possible. The author of this report does not guarantee complete coverage (see disclaimer).

Codebase Submitted for the Audit

The audit has been performed on the following target:

Repository	https://github.com/gnolang/gno
Label	Paths referencing this target are prefixed below with `mempackage_type:`
Commit	`794907c9455aceb31f0abe42dee9b2ae35ffa8a4`
Scope	The scope is restricted to the following files in the `mempackage_type` branch: • `gno/gnovm/pkg/gnolang/alloc.go` • `gno/gnovm/pkg/gnolang/preprocess.go`

	• `gnovm/pkg/gnolang/values_fill.go` • `gnovm/pkg/gnolang/uverse.go:1060-1102`

Repository	https://github.com/gnolang/gno
Label	Paths referencing this target are prefixed below with `pr_4060:`
Scope	The scope is restricted to the changes applied in the following pull requests: • https://github.com/gnolang/gno/pull/4060 reviewed at commit `fd67023b8a8520fffc1a3d5cb54d69b6dfca45f7`, base branch at `a3bffb2eb311cc27c419b519146a9906b5a82dba`.

Fixes verified at commit: `98d135743f32a49c8018a4a2b1ec56b22cc525dd`

Note that only fixes to the issues described in this report have been reviewed at this commit. Any further changes such as additional features have not been reviewed.

Methodology

The audit has been performed in the following steps:

1. Gaining an understanding of the code base's intended purpose by reading the available documentation.
2. Automated source code and dependency analysis.
3. Manual line-by-line analysis of the source code for security vulnerabilities and use of best practice guidelines, including but not limited to:
    a. Race condition analysis
    b. Under-/overflow issues
    c. Key management vulnerabilities
4. Report preparation

Functionality Overview

Gno is an interpreted and fully deterministic implementation of the Go programming language, designed to build succinct and composable smart contracts.

The first blockchain to use it is gno.land, a Proof of Contribution-based chain, backed by a variation of the Tendermint consensus engine.

How to Read This Report

This report classifies the issues found into the following severity categories:

Severity	Description
Critical	A serious and exploitable vulnerability that can lead to loss of funds, unrecoverable locked funds, or catastrophic denial of service.
Major	A vulnerability or bug that can affect the correct functioning of the system, lead to incorrect states or denial of service.
Minor	A violation of common best practices or incorrect usage of primitives, which may not currently have a major impact on security, but may do so in the future or introduce inefficiencies.
Informational	Comments and recommendations of design decisions or potential optimizations, that are not relevant to security. Their application may improve aspects, such as user experience or readability, but is not strictly necessary. This category may also include opinionated recommendations that the project team might not share.

The status of an issue can be one of the following: **Pending**, **Acknowledged**, **Partially Resolved**, or **Resolved**.

Note that audits are an important step to improving the security of smart contracts and can find many issues. However, auditing complex codebases has its limits and a remaining risk is present (see disclaimer).

Users of the system should exercise caution. In order to help with the evaluation of the remaining risk, we provide a measure of the following key indicators: **code complexity**, **code readability**, **level of documentation**, and **test coverage**. We include a table with these criteria below.

Note that high complexity or low test coverage does not necessarily equate to a higher risk, although certain bugs are more easily detected in unit testing than in a security audit and vice versa.

Code Quality Criteria

The auditor team assesses the codebase's code quality criteria as follows:

Criteria	Status	Comment
Code complexity	**High**	The codebase in scope includes the interaction of multiple components, including the gno.land chain, the GnoVM, and Tendermint. The chain incorporates unique concepts and while smart contracts are written in Gno, a new language derived from Go.
Code readability and clarity	**Medium-High**	-
Level of documentation	**Medium**	-
Test coverage	**Medium**	Test coverage reported by `go test` for `mempackage_type:gnovm/pkg/gnolang` is `74.20%`. Test coverage for `pr_4060` cannot be computed due to compilation errors. Gno smart contracts do not provide tooling for computing test coverage.

Summary of Findings

No	Description	Severity	Status
1	Insufficient gas metering and unmetered operations lead to denial of service	Critical	Partially Resolved
2	Pass-by-value parameter in garbage collector nullifies gas accounting	Critical	Resolved
3	`ExpandWith` omits memory accounting for wrapped values	Critical	Resolved
4	Underestimated CPU cycle cost for `doOpReturnAfterCopy`	Major	Acknowledged
5	Unbounded recursion in `Stacktrace` serialization enables DoS	Major	Resolved
6	Type checker bypass for data structures and interfaces	Major	Resolved
7	Incorrect relations between interfaces can cause realm panics	Major	Resolved
8	Garbage collector can exhaust the callstack and crash the node	Major	Resolved
9	Integer overflow in memory allocator enables bypass of resource limits	Major	Resolved
10	Concerns regarding the `DeepFill` method	Minor	Resolved
11	Incorrect early return in `LeftmostX` prevents proper heap escape analysis for nested selector expressions	Minor	Resolved
12	Returning heap pointers across realms may enable phishing via metadata spoofing	Minor	Acknowledged
13	Missing integrity check in exception chaining	Minor	Resolved
14	Usage of undefined `gno.MPStdlib` and related identifiers cause compilation failures	Minor	Acknowledged
15	Lack of transactional isolation in `revive` function	Minor	Acknowledged
16	Usage of the type checker in testing mode	Minor	Resolved
17	Nonce and package identifier are not validated	Minor	Resolved

	during unmarshalling		
18	Inconsistency in the single-linked list implementation	**Informational**	**Resolved**
19	Missing functionality related to the `autoNative` parameter	**Informational**	**Resolved**
20	Redundant code	**Informational**	**Resolved**
21	Ineffective overflow protection	**Informational**	**Resolved**
22	Missing inclusion of `pckgName` field in shallow size allocation calculation	**Informational**	**Resolved**
23	Redundant slice length evaluation in defer stack handling	**Informational**	**Resolved**
24	Use of the magic string "`testing/base`" reduces maintainability	**Informational**	**Resolved**
25	Misleading naming	**Informational**	**Partially Resolved**

Detailed Findings

1. Insufficient gas metering and unmetered operations lead to denial of service

Severity: Critical

The current implementation in multiple areas of the `gnovm` package exhibits issues related to gas metering and resource management, which could lead to system inefficiencies and potential denial of service vulnerabilities:

Fixed gas costs for dynamic operations

In `pr_4060:gnovm/pkg/gnolang/machine.go`, the `Run` method charges gas for each operation by calling `incrCPU` with a fixed number of cycles.

However, this fixed-cost approach does not reflect the variability in computational demand.

For example, the `OpSliceLit2` operation in `pr_4060:gnovm/pkg/gnolang/op_expressions.go:588`, processing slice literals with explicit index assignments has a predetermined gas cost of `467` cycles.

The associated function iterates based on the provided user input. If a user supplies extremely large integer keys for the slice, the system attempts to allocate a slice of length equal to the maximum key plus one.

Without proper bounds or memory checks, this behavior could lead to excessive memory allocation and computational resource usage, potentially causing a denial of service.

Fixed gas fee for frame operations

In `pr_4060:gnovm/pkg/gnolang/machine.go:33`, the `Machine` struct defines `Frames` as a slice of `Frame` structs used to represent the function call stack.

Each `Frame` can vary significantly in memory usage depending on runtime behavior, such as the number of `Defers` it holds or the amount of local context, but the gas cost for pushing a frame (e.g., via `PushFrameCall`) is charged as a fixed, static amount.

This creates a mismatch between gas accounting and actual resource consumption. A lightweight `Frame` and a `Frame` holding many `Defers` or large local structures are charged the same, despite the latter consuming more memory.

This static fee model opens the door to resource abuse, particularly memory exhaustion attacks, while staying within gas limits. For example, pushing many deep or defer-heavy frames allows unbounded memory consumption with only predictable, fixed gas cost.

Unmetered recursion patterns

In `pr_4060:gnovm/pkg/gnolang/machine.go:825`, the `RunStatement` method is responsible for executing a statement. This method retrieves the source from the store, performs preprocessing, configures operations within the machine, and then executes the statement. While most operations in this process are gas-metered, the `Preprocess` phase is not.

Specifically, the `Preprocess` function in `pr_4060:gnovm/pkg/gnolang/preprocess.go:433` calls the `Transcribe` function, which internally executes large recursive operations. These recursive calls are not metered for gas consumption. In scenarios involving complex source inputs with deeply nested structures, these unmetered recursions could become computationally expensive, consuming significant resources and potentially impacting system performance.

Consequently, this omission results in inefficiency for the chain and exposes the system to potential denial of service attacks, as malicious actors could exploit these unmetered operations to overwhelm the system.

Recommendation

We recommend replacing the fixed gas cost model with another model that dynamically accounts for the complexity of each operation:

- Introduce dynamic gas calculations based on input size and the number of iterations required by an operation. Conduct load tests to evaluate the performance of the updated gas metering logic under varying input sizes.
- Implement gas metering for recursive operations by adding gas consumption checks at strategic points. Establish upper limits for recursion depth and object size to mitigate the risk of excessive resource usage.

Status: Partially Resolved

2. Pass-by-value parameter in garbage collector nullifies gas accounting

Severity: Critical

The `GarbageCollect` method, defined in `mempackage_type:gnovm/pkg/gnolang/garbage_collector.go:36-112`, is designed to track the number of visited objects and charge gas accordingly using `visitCount`.

However, it incorrectly calculates gas consumption due to a flaw in how the `visitCount` variable is handled.

Specifically, `visitCount` is passed by value to the `GCVisitorFn` function, resulting in updates being made only to the function's local copy.

As a result, the original `visitCount` used in the gas calculation remains zero, and no gas is consumed for the actual computational effort.

This bypass can be exploited by malicious actors to construct deeply nested or complex structures that incur significant computation at zero gas cost, creating a potential vector for denial of service (DoS) through resource exhaustion.

A test case showcasing the issue is provided in the Appendix.

Recommendation

We recommend passing `visitCount` by reference to ensure that all increments within the visitor closure are reflected in the original variable.

Status: Resolved

3. `ExpandWith` omits memory accounting for wrapped values

Severity: Critical

In `mempackage_type:gnovm/pkg/gnolang/values.go:2461-2490`, the `ExpandWith` function transfers variables from a package's static block to a runtime block when new names are introduced dynamically, such as during interactive evaluation or control flow expansions. These variables originate from preprocessing with placeholder values that occupy no actual memory.

When `ExpandWith` executes, it allocates metadata entries for each new name and, for variables marked to reside on the heap, wraps the placeholder values in heap containers.

However, this wrapping process only incurs a small fixed memory charge for the wrapper itself and the associated metadata. The actual content inside these variables, such as large arrays or complex structs, is not allocated or billed during this step.

As a result, there is a substantial discrepancy between the memory usage reported by the allocator and the true memory consumed.

Consequently, it is possible to allocate large amounts of memory without paying the appropriate gas fees, potentially leading to denial of service (DoS) through resource exhaustion.

A test case showcasing the issue is provided in the Appendix.

Recommendation

We recommend modifying `ExpandWith` to ensure that the full memory cost of all values, including those wrapped in heap containers, is correctly charged to the allocator.

Status: Resolved

4. Underestimated CPU cycle cost for `doOpReturnAfterCopy`

Severity: Major

In `pr_4060:gnovm/pkg/gnolang/machine.go:1048-1049`, both `OpCPUReturn` and `OpCPUReturnAfterCopy` are assigned the same value, specifically `38`, implying equal CPU cycle costs.

However, the `doOpReturnAfterCopy` function involves additional complexity that results in a higher CPU workload compared to `doOpReturn`.

Consequently, assigning them the same CPU cycle cost leads to an underestimation for `doOpReturnAfterCopy`, resulting in mispriced computations and potentially denial of service (DoS).

Recommendation

We recommend updating the associated cost tables or metering logic to ensure accurate accounting of execution costs based on actual computational complexity.

Status: Acknowledged

5. Unbounded recursion in `Stacktrace` serialization enables DoS

Severity: Major

In `pr_4060:gnovm/pkg/gnolang/frame.go:240-242`, the `StringWithStacktrace` function constructs an error message by calling `Stacktrace.String`, which internally invokes `toExprTrace`.

However, the `toExprTrace` function processes expressions recursively without enforcing a maximum recursion depth.

This unbounded recursion can be exploited by a malicious actor crafting deeply nested expressions, leading to excessive memory usage or stack overflow, ultimately resulting in a denial of service (DoS).

Recommendation

We recommend introducing a recursion depth limit within `toExprTrace` to prevent unbounded processing.

Status: Resolved

6. Type checker bypass for data structures and interfaces

Severity: Major

In `mempackage_type:gnovm/pkg/gnolang/types.go:1026-1056`, the `VerifyImplementedBy` function is responsible for ensuring that a given type correctly implements all methods of a specified interface. This process involves iterating over the interface's methods, retrieving corresponding fields from the candidate type, and verifying their names and function signatures.

However, the type validation logic only applies if the retrieved field matches a function type (`FuncType`). There is no accompanying `else` clause to handle non-function fields, resulting in a silent fallback where any data field with a matching name is incorrectly accepted as a valid method.

This oversight leads to fundamental type safety violations. Struct fields that merely share a name with an interface method are treated as valid implementations, even though they lack the correct type or function signature. Such false positives can silently pass compile-time checks and remain undetected until runtime. When the system later attempts to invoke the method, a panic will occur due to the field not being callable, breaking execution flow.

A test case showcasing the issue is provided in the Appendix.

Recommendation

We recommend extending the verification logic to explicitly reject non-function fields during interface checks by implementing the missing `else` clause correspondingly.

Status: Resolved

7. Incorrect relations between interfaces can cause realm panics

Severity: Major

In `mempackage_type:gnovm/pkg/gnolang/type_check.go:1122-1159`, the function `shouldSwapOnSpecificity` determines if the types `t1` and `t2`, passed as the parameters, can be ordered. It returns either `false`, when the `t2` is less specific type than `t1`, or `true`, when `t1` is less specific and `t2` is more concrete.

The function correctly handles the cases involving empty interfaces, which must always be treated as the least specific type.

However, the function does not handle more complex cases of non-empty interfaces and simply returns `false` in line `1136`. While an ordered relation does not always exist between non-empty interfaces, it is still possible.

For instance, if one interface declares all the methods from another interface, the former is contained in the latter, so the latter is the least specific of the two interfaces.

One of the consequences of the incorrect functioning of the function `shouldSwapOnSpecificity` is incorrect type checking of binary expressions. For example, expression `a == b` will crash if `a` is less specific than `b`.

A test case showcasing the issue is provided in the Appendix.

Recommendation

We recommend completing the function `shouldSwapOnSpecificity` by implementing the correct ordering of non-empty interfaces.

Status: Resolved

8. Garbage collector can exhaust the call stack and crash the node

Severity: Major

In `mempackage_type:gnovm/pkg/gnolang/garbage_collector.go:38-115`, the function `GarbageCollect` performs the following operations:

1. Schedules defer functions that perform actual collection and charge gas, in lines `42-52`.
2. Resets the allocator in line `55`, meaning the number of allocated bytes is set to `0`.
3. Increase the number of GC cycles in line `58`.
4. Instantiates the visitor using the function `GCVisitorFn` in line `61`.
5. Applies the visitor to the root objects, which are blocks, frames, and exceptions.

The function `GCVisitorFn`, mentioned in step 4, performs gas-related accounting and calls the function `Allocate` that accounts for the allocation of the visited object and, if the allocator runs out of memory, invokes the callback `alloc.collect` in `mempackage_type:gnovm/pkg/gnolang/alloc.go:124`.

However, the callback `alloc.collect` simply calls the function `GarbageCollect` again, rendering a cycle in the call graph. This results in all steps 1-5 being repeated. If the GC does

not actually progress and release enough memory, the cycle continues until it exceeds the maximum allowed stack depth, which is currently set to `128`. After that, the node will crash.

Even if the GC does not exhaust the call stack, when the GC succeeds, all the deferred functions scheduled during the cycle will execute in reverse order. This can have the following implications:

- Decreased performance due to redundant defers and excessive object collection.
- Potential for invalid state since the defers are executed in reverse order.

Recommendation

We recommend passing the OOM error up to the GC handler instead of the recursive call using the `alloc.collect` callback. In the `GarbageCollect` function, the OOM error can be handled cleanly, potentially by restarting the GC. This would result in a straightforward, explicit loop that allows more control over iterations and has a single finalizer after all the iterations.

Status: Resolved

9. Integer overflow in memory allocator enables bypass of resource limits

Severity: Major

The Gno virtual machine's memory allocator, implemented in `gnovm/pkg/gnolang/alloc.go`, contains integer overflow vulnerabilities across multiple allocation functions, including `AllocateString`, `AllocateListArray`, `AllocateMap`, `AllocateStructFields`, and `AllocateBlock`.

These functions perform unchecked arithmetic when computing memory requirements, such as multiplying element counts by per-item sizes, without verifying for integer overflow.

When large, user-controlled input sizes are passed to these functions, the resulting multiplication can wrap into negative values.

For example, trying to allocate a list with a size just above (`math.MaxInt64 / itemSize`) causes the computed byte size to overflow, leading to a negative or significantly reduced allocation value. This corrupted value is then added to the allocator's internal byte counter before any limit enforcement checks occur, effectively bypassing the virtual machine's memory constraints.

This breaks the core security model of the allocator in several ways:

- Memory limit evasion: Negative accounting values bypass the `maxBytes` enforcement, allowing virtually unlimited allocations.

- Denial of service: Reverting transactions still consume computational resources, making the vulnerability exploitable for DoS attacks.
- Bypass with positive wraparound: Attackers can probe values that overflow to small positive numbers, allowing large memory allocations to appear minimal.

Additionally, in `mempackage_type:gnovm/pkg/gnolang/garbage_collector.go:141-156` the garbage collector relies on a memory usage check on `maxBytes` that fails silently if `curBytes` is negative due to overflow, further compounding the issue and allowing corrupted accounting to propagate undetected.

Recommendation

We recommend implementing strict overflow checks using safe arithmetic or range validation before any multiplication or addition in memory accounting routines.

Status: Resolved

10. Concerns regarding the `DeepFill` method

Severity: Minor

In `mempackage_type:gnovm/pkg/gnolang/values_fill.go`, the `DeepFill` method for various types is defined. Among these definitions, several are related to the types that are supposed to be used only for constant expressions, namely `StringValue`, `BigintValue`, and `BigdecValue`.

These types are not collected by the GC, as it can be observed in `mempackage_type:gnovm/pkg/gnolang/garbage_collector.go:333-343`. The reason for these types being protected from the GC is that they are only used for constant expressions. This is also confirmed by the docs for types `BigintValue` and `BigdecValue`.

In `mempackage_type:gnovm/pkg/gnolang/values_fill.go:3-13`, the `DeepFill` implementations for these types silently return the input value without any modification. While the constant semantics is not violated, calls to these methods can indicate incorrect handling of the constant expressions.

Furthermore, in `mempackage_type:gnovm/pkg/gnolang/values_fill.go:63-69`, for several other types, the `DeepFill` method is not implemented and explicitly panics when called.

Recommendation

We recommend handling constant expressions without the necessity to call the `DeepFill` function, as well as either removing the methods or implementing assertions in the debug builds that the method is not actually called for types `StringValue`, `BigintValue` and `BigdecValue`.

Status: Resolved

11. Incorrect early return in `LeftmostX` prevents proper heap escape analysis for nested selector expressions

Severity: Minor

In `mempackage_type:gnovm/pkg/gnolang/helpers.go:805-816`, the `LeftmostX` function attempts to locate the leftmost expression in a selector or index expression chain.

However, it always returns after the first match (`SelectorExpr`), which results in prematurely returning an intermediate object rather than the true root. For example, in a reference like `&a.b.c`, the function would return `b`, not `a`.

This incomplete traversal leads to incorrect behavior in heap escape analysis, especially when the returned value is cast to a `NameExpr`. In such cases, the actual base object `a` is not marked as escaping, even though a pointer to one of its deeply nested fields `c` is assigned globally.

Recommendation

We recommend modifying the `LeftmostX` function to perform a full recursive descent through `SelectorExpr` layers until a non-selector expression is found. Avoid returning early within the loop.

Status: Resolved

12. Returning heap pointers across realms may enable phishing via metadata spoofing

Severity: Minor

In `mempackage_type:gnovm/pkg/gnolang/machine.go:2253-2257`, when handling `NameExprTypeHeapUse`, the VM resolves the value by obtaining a pointer from the last execution block and marking it as mutable by setting `ro = false`.

In the context of cross-realm function calls, if a function in `RealmB` returns a pointer to a heap object and this pointer is received by `RealmA` (e.g., via cross-call), `RealmA` can then pass this pointer back or to other functions.

While direct mutation of `RealmB`'s heap by `RealmA` may be restricted due to internal ownership checks, this exchange creates a vector for phishing or spoofing.

The returned pointer still references a heap object containing `RealmB`-specific metadata. If `RealmA` sends this pointer to a third party (or back to `RealmB`), the receiving function may trust the pointer based on its structure or metadata, potentially believing it originated internally. This breaks assumptions around object origin, identity, or authority, and opens the door to deception-based attacks (e.g., spoofing trusted realm-specific handles or roles).

Recommendation

We recommend introducing stricter validation or sealing mechanisms for pointers that cross realm boundaries. At a minimum, annotate pointers with their realm of origin and restrict use (e.g., disallow returning a foreign pointer unwrapped).

Status: Acknowledged

13. Missing integrity check in exception chaining

Severity: Minor

In `pr_4060:gnovm/pkg/gnolang/frame.go:260-274`, the `WithPrevious` method allows chaining of two `Exception` instances.

However, the function fails to verify whether `e2.Next` is already set.

Consequently, if `e2` is part of an existing `Exception` chain, this oversight results in the silent corruption of the chain, leading to inconsistent or broken exception handling behavior.

Such a condition may impede accurate backtracking or error propagation in the runtime environment.

Recommendation

We recommend adding a check to verify that `e2.Next` is `nil` before assigning a new link. If `e2.Next` is already populated, the function should either reject the operation with an explicit error or safely detach `e2` from its current chain prior to re-linking.

Status: Resolved

14. Usage of undefined `gno.MPStdlib` and related identifiers cause compilation failures

Severity: Minor

Building the code with `go build ./...` fails due to several undefined identifiers from the gno package. These references appear to rely on symbols that are either no longer part of the gno public API or require a specific branch/version not clearly documented.

The following undefined identifiers are used:

- In `mempackage_type:gno.land/pkg/sdk/vm/keeper.go:204:61`:
  - `gno.MPStdlib` is passed to `ReadMemPackage`.
- In `mempackage_type:gno.land/pkg/sdk/vm/keeper.go:345:20`:
  - `gno.MPAll` is used, but undefined.
- In `mempackage_type:gno.land/pkg/sdk/vm/keeper.go:355:16`:
  - `gno.ValidateMemPackageForType` is called, but not found.
- In `mempackage_type:gno.land/pkg/sdk/vm/keeper.go:355:59`:
  - `gno.MPAll` is again used in combination with the above.
- In `mempackage_type:gno.land/pkg/sdk/vm/keeper.go:370:9`:
  - `gno.ReGnoRunPath` is referenced, but not declared.
- In `mempackage_type:gno.land/pkg/sdk/vm/keeper.go:595:20`:
  - Another use of the missing `gno.MPAll`.
- In `mempackage_type:gno.land/pkg/sdk/vm/params.go:49:37`:
  - `gno.ReRealmPath` is undefined.

Recommendation

We recommend resolving the compile-time issues.

Status: Acknowledged

15. Lack of transactional isolation in `revive` function

Severity: Minor

The `revive` function in `mempackage_type:gnovm/pkg/gnolang/uverse.go:1060-1102` is implemented as a native built-in to simulate a try-catch control flow mechanism.

However, its current design lacks transactional safeguards. When `revive` executes a function block, any state changes performed within it are committed, even if the function panics. This violates atomicity and undermines the reliability of exception-based control flow.

Additionally, the execution fails to enforce the documented constraint that the provided `fn` must always end with a `panic`. This results in state inconsistencies and broken transaction semantics.

Furthermore, if the `Machine.ReviveEnabled` flag is active in production, it globally enables this unsafe behavior. Also, in cases where no panic occurs, revive returns a default `TypedValue` that may lead to logical errors during `nil` checks and type assertions due to its ambiguous nature.

Recommendation

We recommend wrapping revive-executed blocks in transactional isolation, ensuring that all state changes are rolled back if a panic does not occur. Enforce the invariant that fn must always panic at the end, possibly via static or runtime checks. Also, redefine the return behavior to avoid ambiguity when no panic is raised.

Status: Acknowledged

16. Usage of the type checker in testing mode

Severity: Minor

In `mempackage_type:gno.land/pkg/sdk/vm/keeper.go:612`, the `TCLatestRelaxed` type-checking mode is used.

However, according to the inline documentation for `TCLatestRelaxed`, located in `mempackage_type:gnovm/pkg/gnolang/gotypecheck.go:136`, it is intended to be used only for testing.

While it is difficult to predict the specific impact of a generated `gno.mod`, there is a general risk of using incorrect versions of dependencies, potentially containing known exploits.

Recommendation

We recommend using the type checker in production mode.

Status: Resolved

17. Nonce and package identifier are not validated during unmarshalling

Severity: Minor

In `pr_4060:gnovm/pkg/gnolang/ownership.go:61-65`, during the unmarshalling process, `NewTime`(used as a nonce) and `PkgId` are not properly validated.

The `NewTime` field, which is always initialized to `1`, should be explicitly checked to ensure it is greater than zero to prevent unintended or malformed inputs. Similarly, `PkgId`, which represents a package identifier, should be validated to confirm it conforms to the expected 20-byte hash format.

Recommendation

We recommend enforcing explicit validation checks during unmarshalling to ensure `NewTime` is strictly greater than zero and that `PkgId` is exactly `20` bytes in length.

Status: Resolved

18. Inconsistency in the single-linked list implementation

Severity: Informational

In `mempackage_type:gnovm/pkg/gnolang/values.go:673-690`, the `Append` function manages the insertion of elements into a singly linked list used to iterate over values stored in a hashmap. The implementation follows a conventional “head and tail” pattern.

However, when appending to an initially empty list, where the `Head` is `nil`, the inserted item is assigned to both the `Head` and `Tail`. While the list’s size is correctly updated to reflect a single element, this dual assignment introduces a subtle inconsistency: the `Head` is exclusively used for marshalling operations, while the `Tail` is only used during unmarshalling.

Although this divergence currently has no functional impact, it reflects an imprecise handling of the edge case and may introduce confusion or bugs in future modifications.

Recommendation

We recommend explicitly handling the empty list case to maintain consistency in list structure. Furthermore, consider refactoring the append logic to more clearly follow the conventional head–tail insertion pattern.

Status: Resolved

19. Missing functionality related to the `autoNative` parameter

Severity: Informational

In `mempackage_type:gnovm/pkg/gnolang/type_check.go:417`, the `checkAssignableTo` function accepts the parameter `autoNative` of type `bool`.

This parameter is not utilized by the function, however, other functions, namely `checkOrConvertType` and `assertAssignableTo` declare the same parameter `autoNative` and pass it carefully down to `checkAssignableTo`.

There are multiple usages of these functions that provide value for the parameter `autoNative` without the need. A few examples are located on:

- `mempackage_type:gnovm/pkg/gnolang/preprocess.go:2735,2853,4241`
- `mempackage_type:gnovm/pkg/gnolang/type_check.go:589,611,800`

Overall, there are almost a hundred function calls initializing the `autoNative` parameter.

Recommendation

We recommend either implementing the missing functionality or removing the unused parameter `autoNative` and optimizing the calls to `checkOrConvertType`, `assertAssignableTo` and `checkAssignableTo`.

Status: Resolved

20. Redundant code

Severity: Informational

The codebase includes multiple redundant function definitions and parameters:

- In `mempackage_type:gnovm/pkg/gnolang/values.go:2495`, the field `Escaped` of the `RefValue` structure only accepts `true` values.
- In `mempackage_type:gnovm/pkg/gnolang/types.go:1669-1671`, the expression `fv.GetType(nil).Params[0].Type` is used. This pattern is used in

few other locations and looks like a workaround for a past issue. In the current version of the codebase, it can be replaced with a simpler expression `fv.Type.(*FuncType).Params[0].Type`.

- In `pr_4060:gnovm/pkg/gnolang/frame.go:87`, the method `SetWithCross` is defined but is not used by any other functions. There is another function with the same name, defined in `mempackage_type:gnovm/pkg/gnolang/nodes.go:468` and called in `mempackage_type:gnovm/pkg/gnolang/preprocess.go:1394`.
- In `pr_4060:gnovm:gnovm/pkg/gnolang/ownership.go:347`, unused function `GetIsTransient` is defined.
- In `pr_4060:gnovm:gnovm/pkg/gnolang/op_call.go:52`, unused function `getFuncTypeExprFromSource` is defined.

Recommendation

We recommend removing the redundant code.

Status: Resolved

21. Ineffective overflow protection

Severity: Informational

In `mempackage_type:gnovm/pkg/gnolang/garbage_collector.go:43`, the gas amount is calculated as `overflow.Mulp(visitCount*VisitCpuFactor, GasFactorCPU)`.

While `overflow.Mulp` is used, it does not affect the inner multiplication `visitCount*VisitCpuFactor`, which is not protected from overflow and would silently wrap around if it happens. However, given that `VisitCpuFactor` equals `8`, the likelihood of overflow is very low.

Additionally, `GasFactorCPU` is only `1` at the moment, so multiplying any expression by this factor cannot overflow. The `overflow.Mulp` protection is actually redundant in the current version of the codebase.

Recommendation

We recommend examining the possible ranges of factors `VisitCpuFactor` and `GasFactorCPU`, and either simplifying the expression or adding the missing overflow protection.

Status: Resolved

22. Missing inclusion of `pckgName` field in shallow size allocation calculation

Severity: Informational

In `mempackage_type:gnovm/pkg/gnolang/alloc.go:347-364`, the shallow size of a package value is estimated by accumulating the memory usage of its `FNames`, `FBlocks`, and `fBlocksMap` fields. The calculation includes string lengths and pointer sizes as appropriate, and returns the total estimated allocation.

However, the `packagName` field, which is a string and part of the package's shallow structure is not accounted for in this allocation calculation. This omission results in an underestimation of the memory used by the package value.

Recommendation

We recommend including the `pckgName` field in the shallow size computation by adding its length plus `allocPkgName` to the total allocation estimate. This ensures consistency and correctness when calculating memory overhead for package values

Status: Resolved

23. Redundant slice length evaluation in defer stack handling

Severity: Informational

In `mempackage_type:gnovm/pkg/gnolang/frame.go:80-84`, the function conditionally pops the last element from the `fr.Defers` slice if it is non-empty.

However, the current implementation evaluates `len(fr.Defers)` multiple times within the same block. Although slice length retrieval is a low-cost operation, repeated evaluations are unnecessary and reduce code clarity.

This is particularly relevant in performance-sensitive paths such as defer execution during function finalization or panic unwinding, where even minor inefficiencies can accumulate.

Recommendation

We recommend caching the result of `len(fr.Defers)` in a local variable before the conditional block to avoid recomputation and improve readability.

Status: Resolved

24. Use of the magic string "`testing/base`" reduces maintainability

Severity: Informational

In `mempackage_type:gnovm/pkg/gnolang/preprocess.go:1657-1658`, the logic uses a hardcoded string literal "`testing/base`" to identify a specific package path for handling special-case behavior during preprocessing.

It creates maintainability challenges, future renames or refactors of the "`testing/base`" path will silently break this logic unless every dependent usage is updated.

Recommendation

We recommend replacing the hardcoded string with a well-named constant (e.g., `TestingBasePkgPath`).

Status: Resolved

25. Misleading naming

Severity: Informational

Throughout the codebase, multiple instances of misleading names and comments are encountered:

- In `mempackage_type:gnovm/pkg/gnolang/types.go:906-908`, the `InterfaceType` structure declares the field of type `[]FieldType` named `Methods`. It is recommended to rename the field since it can also represent an "embedded interface".
- In `mempackage_type/pkg/gnolang/alloc.go:116`, the function `Allocate` does not allocate anything in the actual memory. It only accounts the memory usage and invokes the GC if necessary.

Recommendation

We recommend renaming the misleading declarations and improving the commentary.

Status: Partially Resolved

Appendix: Test Cases

1. Test case for "[Pass-by-value parameter in garbage collector nullifies gas accounting]"

```go
package gnolang

import (
	"testing"
)

// TestVisitCountBug tests the specific bug where visitCount is passed
by value
// instead of by reference, causing GC gas charges to always be 0
func TestVisitCountBug(t *testing.T) {

	t.Run("VisitCount_PassByValue_Bug", func(t *testing.T) {
		// This test demonstrates that visitCount is not properly
tracked
		// due to pass-by-value parameter in GCVisitorFn

		// Create a machine with a mock gas meter to track gas
consumption
		m := NewMachine("test", nil)
		alloc := NewAllocator(10000000)
		m.Alloc = alloc

		// Create a mock gas meter that tracks all gas consumption
		gasLog := []GasEntry{}
		m.GasMeter = &mockGasMeterWithLogging{log: &gasLog}

		// Create structure that should cause significant GC visits
		depth := 100

		// Create nested structs to force many visits
		fields := alloc.NewStructFields(1)
		fields[0] = TypedValue{T: StringType, V:
StringValue("leaf")}
		deepStruct := alloc.NewStruct(fields)

		for i := 0; i < depth; i++ {
			outerFields := alloc.NewStructFields(1)
			outerFields[0] = TypedValue{V: deepStruct}
```

```
                deepStruct = alloc.NewStruct(outerFields)
            }

            t.Logf("Created nested structure with %d levels", depth)

            // Clear any existing gas log
            gasLog = []GasEntry{}
            m.GasMeter = &mockGasMeterWithLogging{log: &gasLog}

            // Force garbage collection
            left, ok := m.GarbageCollect()

            t.Logf("GC completed: left=%d, ok=%v", left, ok)
            t.Logf("Gas log entries: %d", len(gasLog))

            // Look for GC gas charges
            var gcGasCharges []GasEntry
            var totalGCGas int64

            for _, entry := range gasLog {
                if entry.descriptor == "GC" {
                    gcGasCharges = append(gcGasCharges, entry)
                    totalGCGas += entry.amount
                }
                t.Logf("Gas: %d units for '%s'", entry.amount,
entry.descriptor)
            }

            t.Logf("Found %d GC gas charges totaling %d gas",
len(gcGasCharges), totalGCGas)

            if totalGCGas == 0 {
                t.Logf("BUG: GC gas charges are 0 despite %d object
visits", depth)
                t.Logf("This proves visitCount is not properly tracked
due to pass-by-value bug")
            }
        })

        t.Run("Compare_Expected_vs_Actual_GC_Gas", func(t *testing.T) {
            // Calculate what GC gas SHOULD be vs what it actually is

            depth := 50

            // What SHOULD happen with correct visitCount tracking
```

```go
            expectedVisits := int64(depth) // Approximate minimum visits
for nested structure
            expectedGas := expectedVisits * VisitCpuFactor *
GasFactorCPU

            // Create the actual structure and test
            m := NewMachine("test", nil)
            alloc := NewAllocator(10000000)
            m.Alloc = alloc

            gasLog := []GasEntry{}
            m.GasMeter = &mockGasMeterWithLogging{log: &gasLog}

            // Create nested structure
            fields := alloc.NewStructFields(1)
            fields[0] = TypedValue{T: StringType, V:
StringValue("bottom")}
            deepStruct := alloc.NewStruct(fields)

            for i := 0; i < depth; i++ {
                  outerFields := alloc.NewStructFields(1)
                  outerFields[0] = TypedValue{V: deepStruct}
                  deepStruct = alloc.NewStruct(outerFields)
            }

            // Force GC
            m.GarbageCollect()

            // Check actual gas
            var actualGCGas int64
            for _, entry := range gasLog {
                  if entry.descriptor == "GC" {
                        actualGCGas += entry.amount
                  }
            }

            t.Logf("Analysis of GC gas bug:")
            t.Logf("  Structure depth: %d levels", depth)
            t.Logf("  Expected minimum visits: %d", expectedVisits)
            t.Logf("  Expected GC gas (if working): %d", expectedGas)
            t.Logf("  Actual GC gas charged: %d", actualGCGas)
            t.Logf("  Gas accounting broken: %v", actualGCGas == 0)

            if actualGCGas == 0 && expectedGas > 0 {
                  t.Logf("GC operations consume 0 gas due to visitCount
```

```
bug")
                  t.Logf("  This enables 'free' DoS attacks via
expensive GC operations")
                  t.Logf("  Attackers can create expensive nested
structures without gas costs")
            }
      })
}

// Mock gas meter that logs all gas consumption for analysis
type GasEntry struct {
      amount     int64
      descriptor string
}

type mockGasMeterWithLogging struct {
      log      *[]GasEntry
      consumed int64
      limit    int64
}

func (m *mockGasMeterWithLogging) ConsumeGas(amount int64, descriptor
string) {
      *m.log = append(*m.log, GasEntry{amount: amount, descriptor:
descriptor})
      m.consumed += amount
      if m.limit > 0 && m.consumed > m.limit {
            panic("out of gas")
      }
}

func (m *mockGasMeterWithLogging) RefundGas(amount int64, descriptor
string) {
      m.consumed -= amount
      if m.consumed < 0 {
            m.consumed = 0
      }
}

func (m *mockGasMeterWithLogging) GasConsumed() int64 {
      return m.consumed
}

func (m *mockGasMeterWithLogging) GasConsumedToLimit() int64 {
      if m.limit <= 0 {
```

```go
		return 0
	}
	return m.consumed
}

func (m *mockGasMeterWithLogging) GasRemaining() int64 {
	if m.limit <= 0 {
		return 1000000000 // Unlimited
	}
	remaining := m.limit - m.consumed
	if remaining < 0 {
		return 0
	}
	return remaining
}

func (m *mockGasMeterWithLogging) Limit() int64 {
	return m.limit
}

func (m *mockGasMeterWithLogging) IsOutOfGas() bool {
	if m.limit <= 0 {
		return false
	}
	return m.consumed >= m.limit
}

func (m *mockGasMeterWithLogging) IsPastLimit() bool {
	if m.limit <= 0 {
		return false
	}
	return m.consumed > m.limit
}

func (m *mockGasMeterWithLogging) Remaining() int64 {
	return m.GasRemaining()
}
```

2. Test case for "`ExpandWith` omits memory accounting for wrapped values"

```go
t.Run("ExpandWith_Accounting_Gap", func(t *testing.T) {
		alloc := NewAllocator(1000000)
```

```go
		// Step 1: create PackageNode with no names; create runtime
block with it.
		pn := NewPackageNode("test", "test/path", nil)
		rb := alloc.NewBlock(pn, nil) // Values length 0

		// Record allocator bytes after initial block allocation
(should include block base but we only care delta)
		_, beforeBytes := alloc.Status()

		// Step 2: define bigArray and bigStruct AFTER block
creation so static block grows.
		bigArrayType := &ArrayType{Len: 1000, Elt: IntType}
		bigStructType := &StructType{
			PkgPath: "test",
			Fields: []FieldType{
				{Name: "arr", Type: bigArrayType},
				{Name: "str", Type: StringType},
			},
		}

		pn.Define("bigArray", anyValue(bigArrayType))
		pn.Define("bigStruct", anyValue(bigStructType))
		pn.SetIsHeapItem("bigStruct")

		// Sanity: static block now has 2 names, runtime block has 0
values.
		if int(pn.GetStaticBlock().GetNumNames()) != 2 {
			t.Fatalf("expected 2 names in static block, got %d",
pn.GetStaticBlock().GetNumNames())
		}
		if len(rb.Values) != 0 {
			t.Fatalf("expected runtime block to have 0 values, got
%d", len(rb.Values))
		}

		// Step 3: call ExpandWith to trigger allocation.
		rb.ExpandWith(alloc, pn)

		// Measure cost.
		_, afterBytes := alloc.Status()
		totalCost := afterBytes - beforeBytes
		t.Logf("ExpandWith accounted cost: %d bytes", totalCost)

		// Expected costs using real allocator helper
```

```go
		expectedArrayCost := measure(func(a *Allocator) {
a.AllocateListArray(1000) })
		expectedStructCost := measure(func(a *Allocator) {
			a.AllocateStruct()
			a.AllocateStructFields(2)
		})

		expectedContent := expectedArrayCost + expectedStructCost

		// Accounted: two block items + heap item wrapper
		expectedBlockItemsCost := measure(func(a *Allocator) {
a.AllocateBlockItems(2) })
		heapWrapperCost := measure(func(a *Allocator) {
a.AllocateHeapItem() })
		accountedExpected := expectedBlockItemsCost +
heapWrapperCost

		// Sanity
		if totalCost != accountedExpected {
			t.Errorf("unexpected accounted cost. expected %d, got
%d", accountedExpected, totalCost)
		}

		gap := expectedContent - (totalCost - heapWrapperCost) //
remove wrapper as it replaces struct cost partly
		t.Logf("Expected content cost: %d bytes, accounted cost
(excluding wrapper): %d, gap %d", expectedContent,
totalCost-heapWrapperCost, gap)

		if gap <= 0 {
			t.Fatalf("No gap detected - ExpandWith seems to
allocate full content (unexpected)")
		}
	})
}

// helper that measures allocation cost for a callback using real
allocator logic
func measure(cb func(a *Allocator)) int64 {
	helperAlloc := NewAllocator(math.MaxInt64)
	_, before := helperAlloc.Status()
	cb(helperAlloc)
	_, after := helperAlloc.Status()
	return after - before
}
```

3. Test case for "Type checker bypass for data structures and interfaces"

Example using https://play.gno.land:

```
package hello

import "fmt"

type MyStruct struct {
    doSomething int // only name matches the interface
}

type MyInterface interface {
    doSomething(string) string
}

func test() {
    var x MyInterface = MyStruct{} // error is highlighted in UI, but it
passes compilation
    // var x MyInterface = 123 // does not compile at all

    fmt.Println("hi") // execution reaches this line
}
```

Standard Go, e.g. interpreter on https://go.dev/play, prints the error "cannot use MyStruct{} (value of struct type MyStruct) as MyInterface value in variable declaration: MyStruct does not implement MyInterface (MyStruct.doSomething is a field, not a method)".

4. Test case for "Incorrect relations between interfaces can cause realm panics"

Example using https://play.gno.land:

```
package hello

import "fmt"

type A interface {
    F()
```

```go
}

type B interface {
    F()
    G()
}

type T struct {}
func (T) F() {}
func (T) G() {}

func test() {
    var a A = T{}
    var b B = T{}

    fmt.Println(b == a) // no problem
    fmt.Println(a == b) // panic
}
```

Same code compiles and runs on https://go.dev/play/ without problem.